

Text Summarisation

Condense long text into its key points — two different ways.

Summarisation takes a long piece of text and produces a shorter version that preserves the most important information. There are two fundamentally different approaches.

Extractive vs. abstractive summarisation

Approach	How it works	Example
Extractive	Picks the most important sentences directly from the original text, unchanged	Copies 2-3 key sentences verbatim from a news article
Abstractive	Generates new sentences in different words that capture the meaning	Writes a fresh paragraph in its own words, like a human editor would

Extractive summarisation — a simple approach

Extractive methods often score sentences by importance (e.g. how many frequent/keyword terms they contain) and select the top-scoring ones, keeping their original order.

!pip install sumy

Show hidden output

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

from sumy.parsers.plaintext import PlaintextParser
from sumy.nlp.tokenizers import Tokenizer
from sumy.summarizers.lsa import LsaSummarizer
text = """
"Good morning. I am Rama Chandra, a Computer Science Engineering student with a strong interest in data-driven technologies.
"""

parser = PlaintextParser.from_string(text, Tokenizer('english'))
summarizer = LsaSummarizer()
summary = summarizer(parser.document, 2) # pick top 2 sentences
for sentence in summary:
    print(sentence)
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Over the last few months, I have focused on building my skills in Python, SQL, Machine Learning, and Data Analytics.
I enjoy working on practical problems, learning new technologies, and continuously improving my technical abilities.
```

Start coding or generate with AI.

Abstractive summarisation with HuggingFace

HuggingFace's transformers library provides ready-to-use pipelines for abstractive summarisation, powered by pretrained models like BART or T5. One line of code is often enough.

!pip install transformers torch

Show hidden output

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

article = ""

"Good morning. I am Rama Chandra, a Computer Science Engineering student with a strong interest in data-driven technologies."

tokenizer = AutoTokenizer.from_pretrained("sshleifer/distilbart-cnn-12-6")
model = AutoModelForSeq2SeqLM.from_pretrained("sshleifer/distilbart-cnn-12-6")

inputs = tokenizer(article, return_tensors="pt", truncation=True, max_length=1024)
summary_ids = model.generate(inputs["input_ids"], num_beams=4, max_length=60, min_length=20, early_stopping=True)

print(tokenizer.decode(summary_ids[0], skip_special_tokens=True))

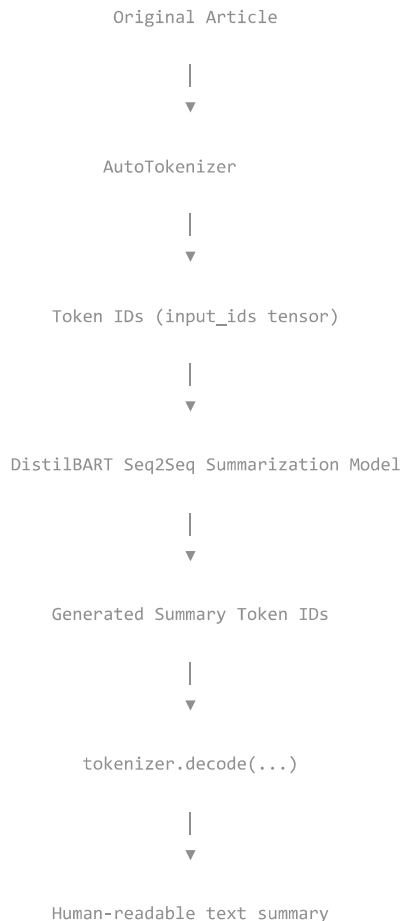
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as the secret `HF_TOKEN` in your Colab secrets, and restart this notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
config.json: 100%          1.80k/1.80k [00:00<00:00, 124kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
tokenizer_config.json: 100%          26.0/26.0 [00:00<00:00, 2.09kB/s]
vocab.json: 100%          899k/899k [00:00<00:00, 24.2MB/s]
merges.txt: 100%          456k/456k [00:00<00:00, 24.2MB/s]
pytorch_model.bin: 100%          1.22G/1.22G [00:22<00:00, 56.1MB/s]
model.safetensors: 100%          1.22G/1.22G [00:14<00:00, 217MB/s]
[transformers] Please make sure the generation config includes `forced_bos_token_id=0`.
Loading weights: 100%          358/358 [00:00<00:00, 1624.51it/s]
Rama Chandra is a Computer Science Engineering student with a strong interest in data-driven technologies . He is currently

```

input->encoder->learns meaning->decode->output sequence



Extractive vs abstractive — which to choose?

✓ Named Entity Recognition (NER)

Named Entity Recognition scans text and labels spans of words that refer to specific real-world entities — people (PERSON), companies (ORG), places (LOC/GPE), dates (DATE), and more.

Example:

Consider the sentence: "Elon Musk founded SpaceX in 2002 in Hawthorne." An NER model would tag it like this:

Text span	Entity type	Meaning
Elon Musk	PERSON	A named individual
SpaceX	ORG	An organisation / company
2002	DATE	A date or year
Hawthorne	GPE / LOC	A geographic location

NER with spaCy

spaCy is a fast, production-ready NLP library with built-in NER models. It also provides a built-in visualiser (displacy) that colour-codes entities directly in the text.

```
!pip install spacy
```

Show hidden output

```
import spacy
nlp = spacy.load('en_core_web_sm')
text = 'the Vamsikumar went for a Secret Date with the special person from nen cheppani college but not ksrm Yesterday at '
doc = nlp(text)
for ent in doc.ents:
    print(ent.text, '->', ent.label_)
```

```
Vamsikumar -> PERSON
a Secret Date -> WORK_OF_ART
Yesterday -> DATE
NakNachina Restaurent -> ORG
```

Colour-coded visualisation with displaCy

```
from spacy import displacy
# In a Jupyter notebook, this renders inline with coloured highlights:
displacy.render(doc, style='ent', jupyter=True)
# To save as an HTML file instead:
# Explicitly set jupyter=False to ensure displacy.render returns the HTML string
html = displacy.render(doc, style='ent', page=True, jupyter=False)
with open('entities.html', 'w') as f:
    f.write(html)
```

```
the  Vamsikumar  PERSON  went for  a Secret Date  WORK_OF_ART  with the special person from nen cheppani college but not ksrm  Yesterday
DATE  at  NakNachina Restaurent  ORG
```

Common spaCy entity labels

Label	Meaning	Example
PERSON	People, including fictional	Marie Curie
ORG	Companies, agencies, institutions	United Nations
GPE	Countries, cities, states	Brazil, Tokyo
DATE	Absolute or relative dates	2002, next Monday
MONEY	Monetary values	\$10 million
PRODUCT	Objects, vehicles, products	iPhone, Boeing 747

Text Translation

From one-line English-to-French to 200+ language multilingual models.

Modern translation models can convert text between languages with near-human fluency, accessible through HuggingFace pipelines in a single line of code.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Load a specific translation model and tokenizer
tokenizer = AutoTokenizer.from_pretrained("Helsinki-NLP/opus-mt-en-hi")
model = AutoModelForSeq2SeqLM.from_pretrained("Helsinki-NLP/opus-mt-en-hi")

text_to_translate = 'Suma loves to eat in hostel for entire life'

# Tokenize the input text
inputs = tokenizer(text_to_translate, return_tensors="pt")

# Generate the translation
translated_ids = model.generate(**inputs)

# Decode the translated IDs back to text
translated_text = tokenizer.decode(translated_ids[0], skip_special_tokens=True)
print(translated_text)
# Expected Output: 'L'apprentissage automatique change le monde.'
```

```
tokenizer_config.json: 100% 44.0/44.0 [00:00<00:00, 887B/s]
source.spm: 100% 812k/812k [00:00<00:00, 11.4MB/s]
target.spm: 100% 1.07M/1.07M [00:00<00:00, 14.5MB/s]
vocab.json: 100% 2.10M/2.10M [00:00<00:00, 16.2MB/s]
Loading weights: 100% 258/258 [00:00<00:00, 4712.82it/s]
सूता पूरे जीवन के लिए होस्टेल में खाने को प्यार करती है
```

Multilingual models

Some models — like Facebook's M2M-100 or NLLB (No Language Left Behind) — support translating between 200+ languages with a single model, by specifying the source and target language codes.

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# A multilingual model that handles many language pairs
model_name = "facebook/nllb-200-distilled-600M"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

text_to_translate = "Good night, how are you?"
src_lang = "eng_Latn"
tgt_lang = "hin_Deva" # Hindi

# Set the source language for the tokenizer
```

```

tokenizer.src_lang = src_lang

# Tokenize the input text
inputs = tokenizer(text_to_translate, return_tensors="pt")

# Generate the translation (FIXED LINE)
translated_ids = model.generate(
    **inputs,
    forced_bos_token_id=tokenizer.convert_tokens_to_ids(tgt_lang)
)

# Decode the translated IDs back to text
translated_text = tokenizer.decode(
    translated_ids[0],
    skip_special_tokens=True
)

print(translated_text)

```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:112: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as the secret `HF_TOKEN` in your Colab secrets, and restart this notebook.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100% 846/846 [00:00<00:00, 34.3kB/s]

tokenizer_config.json: 100% 564/564 [00:00<00:00, 14.5kB/s]

sentencepiece.bpe.model: 100% 4.85M/4.85M [00:02<00:00, 2.66MB/s]

tokenizer.json: 100% 17.3M/17.3M [00:00<00:00, 19.7MB/s]

special_tokens_map.json: 100% 3.55k/3.55k [00:00<00:00, 61.4kB/s]

pytorch_model.bin: 100% 2.46G/2.46G [00:38<00:00, 58.3MB/s]

Loading weights: 100% 512/512 [00:00<00:00, 10529.51it/s]

model.safetensors: 22% 534M/2.46G [00:11<00:38, 50.2MB/s]

generation_config.json: 100% 189/189 [00:00<00:00, 3.57kB/s]
शुभ रात्रि, आप कैसे हैं?

```

**** bold text**Building a simple multilingual FAQ bot**

Combine a small FAQ dictionary with a translation pipeline so users can ask questions and receive answers in their preferred language.

```

from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Load the multilingual model and tokenizer directly
model_name = 'facebook/nllb-200-distilled-600M'
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

faq = {
    'hours': 'We are open from 9 AM to 6 PM, Monday to Saturday.',
    'returns': 'You can return any item within 30 days with a receipt.',
    'shipping': 'Standard shipping takes 3-5 business days.',
}

def answer_faq(question_key, target_lang_code):
    english_answer = faq.get(question_key, 'Sorry, I do not have that answer.')

    # Set the source language for the tokenizer
    tokenizer.src_lang = 'eng_Latn'

    # Tokenize the input text
    inputs = tokenizer(english_answer, return_tensors="pt")

    # Generate the translation, explicitly setting the target language
    translated_ids = model.generate(**inputs, forced_bos_token_id=tokenizer.convert_tokens_to_ids(target_lang_code))

    # Decode the translated IDs back to text
    translated_text = tokenizer.decode(translated_ids[0], skip_special_tokens=True)
    return translated_text

# Example: ask about shipping, answer in Spanish
print(answer_faq('hello', 'tel_Telu'))

```

Loading weights: 100%

512/512 [00:00<00:00, 11815.85it/s]

క్షమించండి, నాకు ఆ సమాధానం లేదు.